

- **Multistage Docker build**
- **Docker Compose**

1. Multistage Docker build

Multi-stage builds are a powerful **Docker feature** that allows us to use **multiple FROM statements** in a **single Dockerfile** to create **optimized, minimal, and secure production images**. This technique **separates the build environment**, which often requires large compilers and SDKs, **from the final, lean runtime environment**.

How They Work

Multiple Stages: A single Dockerfile includes multiple **FROM** instructions, each beginning a **new build stage**.

Naming Stages: Stages can be named using **AS <NAME>** (e.g., **FROM golang:1.22-alpine AS builder**) for easier reference.

Selective Copying: The key is the **COPY --from=<stage-name-or-number>** instruction. This command copies only the necessary artifacts (like a **compiled binary or production-ready files**) from an **earlier stage** into a later, typically **much smaller**, stage.

Final Image: The resulting image is based solely on the **last stage** defined in the Dockerfile, discarding all **intermediate stages and their unnecessary build tools and dependencies**.

Benefits

Smaller Image Sizes: Significantly reduces the final image size by **leaving behind build-time dependencies**, leading to faster transfer and deployment times.

Enhanced Security: A smaller image has a reduced "attack surface" because it contains fewer packages and tools that could have vulnerabilities.

Simplified Dockerfiles: Consolidates the entire build process into a single Dockerfile, eliminating the need for separate build scripts or multiple Dockerfiles (e.g., one for development and one for production).

Improved Caching: Docker can cache intermediate layers more effectively, speeding up subsequent builds, especially when source code changes frequently but dependencies do not.

Example: Go Application

A common use case is with compiled languages like Go or Java, where the compiler is not needed at runtime

Dockerfile

#Stage 1: Build the application with the full SDK

FROM golang:1.22-alpine AS builder WORKDIR /app COPY go.mod go.sum ./ RUN go mod download COPY . .

Compile the binary

RUN CGO_ENABLED=0 GOOS=linux go build -o /app/server ./cmd/server

#Stage 2: Create a minimal runtime image

FROM scratch WORKDIR /app

Copy only the compiled binary from the 'builder' stage

COPY --from=builder /app/server . EXPOSE 8080 ENTRYPOINT ["/app/server"]

To build this image, you simply run the standard build command:

`docker build -t my-app:latest ..`

Docker will automatically build both stages but will only produce an image from the final FROM scratch stage containing just the binary. You can also specify a target stage during the build process using the `--target` flag (e.g., `docker build --target builder -t my-app:build .`) for testing or debugging specific stages.

2. Docker Compose

Docker Compose is a **tool** for **defining and running multi-container Docker applications** using a **single YAML configuration file**, typically named `compose.yaml`. This **simplifies** the **management of application stacks** (like a **web service and a database**) that **require multiple containers** to run and communicate with each other.

Key Concepts

Declarative Configuration: Instead of using **multiple, complex docker run commands**, you **define all your services, networks, and volumes in one file**. This file acts as a **blueprint**, ensuring **consistency** across different environments (development, testing, production).

Service Definition: The services section in the Compose file defines each container that makes up our application. We can **specify the image** to use (or a Dockerfile to build from), exposed ports, volumes, and environment variables.

Single Command Execution: A single command, **docker compose up –build** (builds and starts), **docker compose** creates and starts all the defined services and their dependencies.

Networking: By default, Compose **creates a single network** for your **application**, allowing **containers to communicate** with each other using their **service names**.

Common Commands

Docker Compose provides commands for managing the full lifecycle of our application stack:

docker compose up: Builds (if necessary), creates, starts, and attaches to containers for a service. Use the -d flag to run in the background (detached mode).

docker compose down: Stops and removes containers, networks, and, if specified with --volumes, the volumes created by up.

docker compose ps: Lists all containers in the current project, showing their state and port mappings.

docker compose logs: Views the log output from containers.

docker compose up --build: Builds images for your services